

Vesti Transpiler User Manual

Sungbae Jeong

September 24, 2026

Contents

1	Introduction	1
2	Structure of Vesti File	2
3	Keywords	3
3.1	<code>docclass</code> keyword	3
3.2	<code>importpkg</code> keywords	3
3.3	<code>startdoc</code> keyword	3
3.4	<code>useenv</code> keyword	3
3.5	<code>beginv</code> and <code>endenv</code> keywords	4
3.6	<code>defun</code> keyword	4
4	Builtins	5
4.1	<code>#label</code> builtin	5
4.2	<code>#eq</code> builtin	5
4.3	<code>#chardef</code> builtin	6
4.4	<code>#mathchardef</code> builtin	6
4.5	<code>#enum</code> and <code>#enum_counter</code> builtins	6
4.6	<code>#get_filepath</code> builtin	7
4.7	<code>#at_on</code> and <code>#at_off</code> builtins	8
4.8	<code>#ltx3_on</code> and <code>#ltx3_off</code> builtins	8
4.9	<code>#textmode</code> and <code>#mathmode</code> builtins	8
4.10	<code>#tabular</code> and <code>#longtabular</code> builtins	9
4.10.1	A complete short table	9
4.10.2	Options, widths, and alignment	10
4.10.3	Merges and styled borders	11
4.10.4	Border controls and ranges	12
4.10.5	Long tables and page breaks	13
5	Lua API	16
6	Source Code of This Document	17

1 Introduction

I used to create several documents using L^AT_EX (or plain T_EX but T_EX is quite cumbersome to write—especially when working with very complex tables or inserting images). Its markdown-like syntax is also not comfortable to use. For example, here is a simple L^AT_EX document:

```
1 % coprime is my custom class. See https://github.com/e0328eric/coprime.
2 \documentclass[tikz, geometry]{coprime}
3
4 \settitle{My First Document}{Sungbae Jeong}{}
5 \setgeometry{a4paper, margin = 2.5cm}
6
7 \begin{document}
8 \section{Foo}
```

```

9 Hello, World!
10 \begin{figure}[ht]
11     \centering
12     \begin{tikzpicture}
13         \draw (0,0) -- (1,1);
14     \end{tikzpicture}
15 \end{figure}
16
17 The code above is a figure using TikZ.
18
19 \end{document}

```

What annoys me most when using it is the ‘`\begin`’ and ‘`\end`’ blocks. Is there a way to write something much simpler? This question led me to start this project. Currently, the following code is generated into the $\text{\LaTeX}$ code above (except comments) using `vesti`:

```

1 % coprime is my custom class. See https://github.com/e0328eric/coprime.
2 \docclass coprime (tikz, geometry)
3
4 \settitle{My First Document}{Sungbae Jeong}{}
5 \setgeometry{a4paper, margin = 2.5cm}
6
7 \startdoc
8
9 \section{Foo}
10 Hello, World!
11 \useenv figure [ht] {
12     \centering
13     \useenv tikzpicture {
14         \draw (0,0) -- (1,1);
15     }
16 }
17
18 The code above is a figure using TikZ.

```

Everywhere in this paper, the symbol $\sqcup$ denotes a “space” when one use `Vesti`.

2 Structure of `Vesti` File

`Vesti` is similar as $\text{\LaTeX}$. Its structure consists with two parts: `preamble` and `main`. Preamble is the place where $\text{\LaTeX}$ documentclass, packages, and several settings are located. Main body is where actual documentation is located. Below figure is the simple `Vesti` documentation.

```

1 \docclass article (10pt)
2 \importpkg {
3     geometry (a4paper, margin=2.2cm)
4 }
5 \startdoc
6 Hello, Vesti!

```

We will see later, but the very difference with $\text{\LaTeX}$ is that `Vesti` has its own keywords (keywords are colored with purple). It makes the code readable and it is easier and faster to write the document. The keyword `startdoc` splits the preamble and the main part of the documentation similar with

`\begin{document}` in $\text{\LaTeX}$. However, `Vesti` does not have the analogous part of `\end{document}`, because almost every $\text{\LaTeX}$ document (99.999% I’m sure) does not have any code below `\end{document}`. For this reason, `Vesti` automatically ends document when EOF (End Of File) is found.

3 Keywords

Followings are reserved as keywords. In this document, every Vesti keyword has the form like **this**.

beginv	defenv	defun	docclass
endenv	importmod	importpkg	importves
startdoc	useenv		

Table 1: Keywords in Vesti

3.1 **docclass** keyword

Keyword **docclass** is an analogous of `\documentclass` in $\text{\LaTeX}$. If **docclass** is in the main paragraph, it acts just a normal word. In other words, **docclass** activates only in the preamble. The syntax of **docclass** is following:

docclass $\square$ <class name> $\square$ (<arguments>)

Here, arguments are separated by commas and embraced by (). Here are some examples.

- **docclass** $\square$ article
- **docclass** $\square$ article $\square$ (10pt)
- **docclass** $\square$ article $\square$ (10pt, $\square$ twocols)
- **docclass** $\square$ article $\square$ (10pt,twocols)

3.2 **importpkg** keywords

Keyword **importpkg** is an analogous of `\usepackage` in $\text{\LaTeX}$. If **importpkg** is in the main paragraph, it acts just a normal word. In other words, **importpkg** activates only in the preamble.

importpkg has two different syntax. First one is same as **docclass**.

importpkg $\square$ <pkg-name> $\square$ (arguments)

Here, arguments are separated by commas and embraced by (). In the practical case, one should include several packages with options. **importpkg** also supports such case. We will look at an example instead of giving rigorous grammar.

```
1 importpkg {  
2     amsmath, amssymb, amsthm,  
3     geometry (a4paper, margin=2.2cm),  
4 }
```

As one can see, inside of {}, several packages can be used together with their options.

3.3 **startdoc** keyword

Keyword **startdoc** tells to Vesti that the main document starts. In the main document, you can also write **startdoc** in the main document. In that case, **startdoc** does nothing.

3.4 **useenv** keyword

As the name implies, keyword **useenv** is an analogous of `\begin{...}` and `\end{...}` pair in $\text{\LaTeX}$. The simplest **useenv** is like this.

```
useenv center {  
    Hello, World!    or    useenv center { Hello, World! }  
}
```

As you can see, **useenv center** is the part of `\begin{center}`, and the single } is the part of `\end{center}`. Since Vesti knows their pair, one can write a code with several environment, and each pair is properly matched. For instance, above example is written in Vesti like follows. Here, `\useenv` just prints **useenv** in that style.

```

1 useenv figure [ht] {
2   \centering
3   useenv tikzpicture {
4     useenv scope {
5       \path (0,0) node {\vbox{
6         %#\hbox{\tt\useenv center \{}
7         %#\hbox{\tt\obeyspaces Hello, World!}
8         %#\hbox{\tt\obeyspaces\}}
9       }
10    }
11    \path (2.3,0) node {or};
12    useenv scope [shift={(6,0)}] {
13      \path (0,0) node {\tt\useenv center \{ Hello, World! \}};
14    }
15  }
16 }

```

Full syntax about **useenv** is the following.

useenv $\sqcup$ $\langle$ environment name $\rangle \sqcup \langle$ argument $\rangle^* \sqcup \{ \langle$ body $\rangle \}$

where “*” means that the number of $\langle$ argument $\rangle$ is zero or at least one, and

$$\langle \text{argument} \rangle = \begin{cases} (\langle \text{argument} \rangle) & \text{mandatory arguments} \\ [\langle \text{argument} \rangle] & \text{optional arguments} \end{cases}$$

For instance, below one is a valid Vesti code (environment **foo** is undefined in general). As one can see, spaces cannot exist in between $\langle$ argument $\rangle$ s.

```

1 useenv foo (asd)(fff)[\ames and \awdsa](askws)[\rrsaa] {
2   foobar
3 }

```

3.5 **beginv** and **endenv** keywords

As the name implies, both keywords **beginv** and **endenv** are analogous of $\backslash\text{begin}\{\dots\}$ and $\backslash\text{end}\{\dots\}$ pair in $\text{\LaTeX}$, respectively. Thus below code

```

1 beginv center
2   asdsad
3 endenv

```

is exactly same as

```

1 useenv center {
2   asdsad
3 }

```

Then why we need **beginv** and **endenv** if we already have **useenv**? The only reason which both **beginv** and **endenv** exist is defining new environment. For instance, one defines a new environment like following.

```

1 defenv foo {beginv minipage(\textwidth)}{endenv}

```

3.6 **defun** keyword

defun is the keyword which makes a $\text{\LaTeX}$ function. Internally, it uses **xparse** package which is automatically included from modern $\text{\LaTeX}2\epsilon$ kernel. The grammar of **defun** is the following.

defun $\sqcup [\langle$ attr $\rangle] ? \sqcup \langle$ command-name $\rangle \sqcup (\langle$ param-spec $\rangle) ? \sqcup \{ \langle$ implementation $\rangle \}$

Here, `~?` means that this is an optional.

First, let us see some examples for `defun`, and then explain the meaning of each fields.

```
1 \defun foo {Hello, World!}
```

4 Builtin

Vesti also has its own builtin functions, which are prefixed with `#`. One might wonder what distinguishes builtins from keywords. In fact, from the compiler’s internal perspective, there is no real difference. However, in actual language usage, constantly typing the prefix can be somewhat tedious, especially for functions that are commonly used.

From the perspective of language design sometimes desirable to use names that can not be reserved as keywords. For example, Vesti provides a built-in function `#label`, which will be explained later. Since Vesti is a typewriting-oriented language, the word “label” is often used in its ordinary sense rather than in its special semantic meaning within the language.

Followings are reserved as builtin functions.

<code>#at_off</code>	<code>#at_on</code>	<code>#chardef</code>	<code>#copy_file</code>	<code>#def</code>
<code>#engine_type</code>	<code>#enum</code>	<code>#enum_counter</code>	<code>#eq</code>	<code>#get_filepath</code>
<code>#include</code>	<code>#label</code>	<code>#longtabular</code>	<code>#ltx3_off</code>	<code>#ltx3_on</code>
<code>#mathchardef</code>	<code>#mathmode</code>	<code>#noltx3</code>	<code>#picture</code>	<code>#raw_tex</code>
<code>#showfont</code>	<code>#tabular</code>	<code>#textmode</code>	<code>#undef</code>	

Table 2: Builtins in Vesti

Since `fancyvrb` does not allow verbatim-like commands inside of `Verbatim` environment, builtins are not colored in example codes.

4.1 `#label` builtin

For the compiling issue, if one label some environment, one should write like follows using $\LaTeX$ function `\label`.

```
1 useenv theorem { \label{eq:1}
2   1 + 1 = 2.
3 }
```

However, such code is not natural. Thus `#label` comes into the place. One can label environments using `#label` builtin like this.

```
1 #label(eq:1) useenv theorem {
2   1 + 1 = 2.
3 }
```

One can also write like the following, which I personally prefer.

```
1 #label(eq:1)
2 useenv theorem {
3   1 + 1 = 2.
4 }
```

4.2 `#eq` builtin

`#eq` is a abbreviation of ‘`useenv` equation’. Actually, below codes are same.

```
1 #label(eq:1)
2 useenv equation {
3   \sum_{n=1}^{\infty} {1//n^2} = {\pi^2//6}.
4 }
```

```

1 #eq(eq:1) {
2   \sum_{n=1}^{\infty} {1//n^2} = {\pi^2//6}.
3 }

```

Since Vesti is used in the mathematical documents mainly, small syntactic sugar is needed, so Vesti introduces `#eq` builtin.

4.3 `#chardef` builtin

`#chardef` defines a text token using the unicode codepoint. Here is the grammar of `#chardef` builtin.

$$\text{\#chardef} \quad \langle \text{unicode-codepoint} \rangle \quad \langle \text{function} \rangle$$

Here, $\langle \text{unicode-codepoint} \rangle$ must be hexadecimal.

4.4 `#mathchardef` builtin

`#mathchardef` defines a math token using the unicode codepoint. Here is the grammar of `#mathchardef` builtin.

$$\text{\#mathchardef} \quad \langle \text{kind} \rangle \quad \langle \text{font-num} \rangle \quad \langle \text{unicode-codepoint} \rangle \quad \langle \text{function} \rangle$$

Here, $\langle \text{unicode-codepoint} \rangle$ must be hexadecimal. Before explaining each parameter, introduce some examples.

- `#mathchardef \opening 0 29d8 \lfoo`
- `#mathchardef \ordinary 0 2202 \diff`

4.5 `#enum` and `#enum_counter` builtins

`#enum` builtin is just an enumerate environment with minimal support of `enumitem` feature. For example,

```

1 #enum {
2   \item aaa
3   \item bbb
4   \item ccc
5 }

```

prints

1. aaa
2. bbb
3. ccc

`#enum` also can be nested like follows.

```

1 #enum {
2   \item aaa
3   \item #enum {
4     \item bbb
5     \item ccc
6   }
7   \item ddd
8 }

```

It prints

1. aaa
2. (a) bbb

(b) ccc

3. ddd

To customize a format of `\item` like `enumitem`, one can write like the following:

```
1 #enum (\Roman*. ) {  
2   \item aaa  
3   \item #enum ({\roman*}**) {  
4     \item bbb  
5     \item ccc  
6   }  
7   \item ddd  
8 }
```

I. aaa

II. i* bbb
 ii* ccc

III. ddd

As one can see, `#enum` changes the format with inside of `()`. In Vesti, single `*` changes into `{<enum count>}` (including braces), and consecutive `**` changes into single `*` character. Thus, `\Roman*.` changes into `\Roman{enumi}.` and `{\roman*}**` changes into `{\roman{enumii}}*` (because this part is inside of another `#enum`).

Beware that by the parting mechanism, `\roman***` changes into `\roman*{<enum count>}`, and `\roman{*}**` into `\roman*{<enum count>}`, which are invalid arguments for `\roman` L^AT_EX internal command.

`#enum_counter` builtin is a helper to get the name of the current enum counter. For example, if one wants to start a second enum from 3, one can write

```
1 #enum {  
2   \item aaa  
3   \item #enum {  
4     \setcounter{#enum_counter}{2}  
5     \item bbb  
6     \item ccc  
7   }  
8   \item ddd  
9 }
```

and it prints

1. aaa
2. (c) bbb
 (d) ccc
3. ddd

4.6 `#get_filepath` builtin

Because of the Vesti internal, if one includes picture by using `\includegraphics` for instance, providing the raw path cause an error from L^AT_EX with “file not found”. `#get_filepath` builtin adjust raw file path into the new relative one which L^AT_EX knows. Here is the example code.

```
1 \includegraphics{#get_filepath(./foo.png)}
```

Internally, `#get_filepath` changes file path into the relative one with respect to `.vesti-dummy`. For example, if `foo.png`, `foo.ves` and `.vesti-dummy` are located in the root directory, then inside of `foo.ves`, `#get_filepath(./foo.png)` will be changed into `../foo.png` because for `.vesti-dummy`, `foo.png` is located outside of it.

4.7 #at_on and #at_off builtins

Both builtins are same as $\text{\LaTeX}$ functions `\makeatletter` and `\makeatother` plus changes Vesti lexer such that `@` character is also can be a $\text{\LaTeX}$ function name. For instance, without using `#at_on` command, below code does not compiles.

```
1 defun @foo (#1) {Hello, #1!}
```

This is because `defun` expects a valid $\text{\LaTeX}$ function name but the character `@` is not a valid one except after the `\makeatletter` function. Since `defun` is a Vesti grammar, there is no way to tell Vesti that `@` is a right one although one uses `\makeatletter`. By using `#at_on`, below code then compiles.

```
1 #at_on
2 defun @foo (#1) {Hello, #1!}
3 #at_off
```

One can not using `#at_off`, but I strongly recommend to match pairs.

4.8 #ltx3_on and #ltx3_off builtins

Those builtins are similar with `#at_on` and `#at_off` pairs but for $\text{\LaTeX}3$.

4.9 #textmode and #mathmode builtins

Internally, vesti tracks so called `text mode` and `math mode`. In the `math mode`, for instance, the token `->` is changed into `\rightarrow` inside of the `math mode`. The idea is that Vesti changes `->` into `\to`, and so on. But when defining some function using `defun`, for example, one might want to define

```
1 useenv foo {-><-}
```

so that `$_foo$` makes `\rightarrow\leftarrow`, one should change the mode. Here, `#mathmode` comes into the place. One should write instead that

```
1 useenv foo {#mathmode{-><-}}
```

4.10 #tabular and #longtabular builtins

These builtins make tables directly from T_EX boxes, dimensions, glue, and rules. They do not import any L^AT_EX table, color, or graphics package. #tabular produces one indivisible table; #longtabular continues a table across dedicated pages with repeated headers and footers. Both support paragraph cells, rectangular merges, colored backgrounds, and six border patterns. Existing useenv tabular code keeps its usual L^AT_EX meaning.

4.10.1 A complete short table

The table body contains explicit rows and cells. This example includes a spanning title and two colored cells. Its output appears below the listing.

```
1 #tabular(width=300pt, grid=all, padx=5pt,
2         pady=4pt, minheight=24pt) {
3   columns {
4     col(width=flex(1), align=center);
5     col(width=flex(1), align=center);
6     col(width=flex(1), align=center);
7   }
8   body {
9     row { cell(colspan=3) { AT\&T Common Stock } }
10    row {
11      cell { Year }
12      cell(background=cmyk(0,.6,.1,0)) { Price }
13      cell { Dividend }
14    }
15    row {
16      cell(background=cmyk(.6,.1,0,0)) { 1971 }
17      cell { 41--!54 } cell { \$2.60 }
18    }
19    row { cell { 1971 } cell { 41--!54 } cell { \$2.60 } }
20  }
21 }
```

AT&T Common Stock		
Year	Price	Dividend
1971	41-54	\$2.60
1971	41-54	\$2.60

Write columns first and specify width on every col. There must be exactly one nonempty body section. A short table accepts only that section. Write cell {} for an intentional empty cell. Newlines and indentation outside cell content have no structural meaning. An ampersand does not separate cells, and a double backslash does not end a row. Use ordinary escaped text such as \& and \\$ inside a cell.

Vesti's comment syntax still applies: -- starts a comment. For a numeric range, write 41--!54 or use a literal en dash. The --! token produces the en dash shown in the example.

4.10.2 Options, widths, and alignment

Options have the form (key=value, key=value); a trailing comma is allowed. Unknown or duplicate keys, an empty option list, and values of the wrong type are errors. The following reference table is itself a native table.

Scope	Options and defaults
Both builtins	width=natural, align=center, grid=none, padx=4pt, pady=3pt, minheight=0pt, valign=middle, plus border defaults.
Long table	Requires finite width and pageheight; every column must have a fixed or flex width.
Column	Required width; align=left; optional valign, wrap, and before.
Row	Optional minheight, background, and before. Minimum height inherits from the table.
Cell	colspan=1, rowspan=1; optional align, valign, wrap, background, and before.

Choosing widths. A column width is **natural**, a positive dimension such as 35pt, or **flex**(positive weight). Natural columns measure unwrapped content; fixed and flex columns wrap by default. Set **wrap=yes** or **wrap=no** to override this behavior. A wrapped span must include at least one fixed or flex column so its width can be resolved before paragraph typesetting.

The table's width is **natural**, a positive dimension, or the register `\hsize` or `\vsize`. Flex columns require a finite table width and divide the remaining space in proportion to their weights. For example, weights 1 and 2 share the available space in a ratio of one to two. With no flex columns, an explicit table width must equal the total resolved width within one scaled point; it does not add extra space around fixed columns.

The **pageheight** option is available only on `#longtabular`. It accepts a positive dimension or `\hsize` or `\vsize`; **natural** and **flex** are not valid page heights.

A column track includes left and right padding. Vertical borders occupy additional lanes. A **colspan** includes the internal lanes it replaces. Double borders reserve space for both strokes and their gap. If the target width leaves no positive content width, or an unwrapped cell is too wide, the renderer reports an error instead of enlarging the table silently.

Alignment and local styling. Horizontal alignment is **left**, **center**, or **right**. The table's **align** positions the entire table. Cells inherit alignment and wrapping from their starting column. Vertical alignment is **top**, **middle**, or **bottom**. A cell uses its explicit value, then its starting column's value, then the table default. Minimum height is a lower bound and never clips content.

Use **before={\bf}** for a local font change. Hooks run in this order: starting column, row, cell, then cell content. A cell uses its own background, then its originating row's background; if neither supplies one, it is transparent. A **rowspan** retains that background across its complete rectangle.

Dimensions and colors. Dimensions are unsigned decimals immediately followed by a unit: **pt**, **pc**, **in**, **bp**, **cm**, **mm**, **dd**, **cc**, **sp**, **em**, or **ex**. For example, `.6pt` and `3em` are valid; signs, exponent notation, expressions, and arbitrary macros are not. Registers and font-relative units are resolved at table entry, before cell font hooks. Widths, page height, stroke thickness, explicit gaps, and dash lengths must be positive. Padding, minimum heights, and pattern phase may be zero.

Colors are **gray**(g), **rgb**(r,g,b), or **cmyk**(c,m,y,k), with every component between zero and one. There are no named colors or transparency values in the table grammar.

4.10.3 Merges and styled borders

Use `colspan` and `rowspan` with positive integer values to merge a rectangle. Cells are placed in the next free column from left to right. Later rows omit positions already occupied by a `rowspan`. Spans cannot intersect or cross a section boundary, and all remaining positions must be filled.

The `grid` option accepts `none`, `frame`, `rows`, `columns`, or `all`. The `rows` and `columns` values draw only interior boundaries; `frame` draws the perimeter, and `all` draws both. Automatic borders inside merged cells are suppressed. Explicit rules must also avoid the interior of a merge.

This example uses all six border patterns. The partial dotted rule covers only columns two and three, so it leaves the `rowspan` intact.

```
1 #tabular(width=340pt, grid=all, pady=4pt, minheight=24pt) {
2   columns {
3     col(width=flex(1), align=center);
4     col(width=flex(1), align=center);
5     col(width=flex(2));
6   }
7   body {
8     hline(style=solid, thickness=1pt, color=rgb(.1,.2,.4));
9     row { cell(colspan=3) { Merges and border patterns } }
10    hline(style=double, thickness=.7pt, gap=1pt,
11          color=rgb(.1,.3,.6));
12    row {
13      cell(rowspan=2, background=rgb(.9,.95,1)) { Group A }
14      cell { Dotted }
15      cell { A partial rule below this row. }
16    }
17    hline(from=2, to=3, style=dotted, thickness=.7pt,
18          gap=1.2pt, color=rgb(.65,.15,.2));
19    row { cell { Dashed } cell { The merged cell continues. } }
20    hline(style=dashed, dashlength=4pt, gap=2pt,
21          color=rgb(.1,.45,.3));
22    row { cell { B } cell { Dash-dot } cell { Separate marks. } }
23    hline(style=dashdot, dashlength=4pt, gap=1.5pt,
24          phase=.5pt, color=rgb(.35,.2,.6));
25    row { cell { C } cell { Dash-dot-dot } cell { Square dots. } }
26    hline(style=dashdotdot, dashlength=4pt, gap=1.5pt,
27          color=rgb(.6,.3,.1));
28    vline(after=0, style=double, thickness=.7pt, gap=1pt);
29    vline(after=3, style=solid, thickness=1pt);
30  }
31 }
```

Merges and border patterns		
Group A	Dotted	A partial rule below this row.
	Dashed	The merged cell continues.
B	Dash-dot	Separate marks.
C	Dash-dot-dot	Square dots.

4.10.4 Border controls and ranges

Table-level defaults apply to automatic grid edges and are inherited by explicit rules. A rule can override each component independently.

Table default	Rule option	Meaning
<code>rulestyle=solid</code>	<code>style</code>	Pattern: solid, double, dotted, dashed, dashdot, dashdotdot.
<code>rulewidth=.4pt</code>	<code>thickness</code>	Width of one stroke.
<code>rulecolor=gray(0)</code>	<code>color</code>	Gray, RGB, or CMYK color.
<code>rulegap=auto</code>	<code>gap</code>	Clear space between marks or double strokes.
<code>ruledashlength=auto</code>	<code>dashlength</code>	Length of each dash.
<code>rulephase=0pt</code>	<code>phase</code>	Distance into the repeating pattern at a run's start.

Horizontal and vertical rules. `hline`; draws a rule at its position between rows, or before the first or after the last row. Optional `from` and `to` select a one-based, inclusive column range; omission means the full width. `vline(after=0)`; selects the left edge. The required `after` numbers column boundaries from zero through the number of columns. Its optional `from` and `to` select a one-based, inclusive row range in the current section. A vertical rule's source position does not select its rows. Omitting both range endpoints selects every row in the current section.

Explicit rules replace automatic grid edges. Identical explicit overlaps merge; incompatible overlaps are errors, including at header/body/footer joins. Do not draw a full-width horizontal rule through a `rowspan`: select only its exposed columns as in the dotted example above.

Pattern dimensions. For final stroke thickness t , `gap=auto` means t for double lines and $2t$ for repeating patterns. `dashlength=auto` means $4t$. A double border reserves $2t + g$, where g is its clear gap; all other patterns reserve t . Changing thickness or a double gap can change layout and page fits. Changing color, dash spacing, or phase alone does not change the reserved width. An explicit local `auto` resets an inherited numeric value.

On an individual rule, `gap` applies to nonsolid styles; `dashlength` applies only to dashed, dashdot, and dashdotdot; `phase` applies only to repeating patterns. Inapplicable explicit controls are errors. Table defaults may remain unused until a matching style uses them.

Dots are square rules, not font characters. Dash-dot repeats dash, gap, dot, gap; dash-dot-dot adds one more dot and gap. Patterns run left to right or top to bottom, normalize phase modulo the full period, and clip at their endpoints. Equal adjacent borders form one continuous run across ordinary row/column junctions. A merge interruption, style change, or page cut starts a new run. Clear gaps are transparent; horizontal ink paints over vertical ink at crossings.

4.10.5 Long tables and page breaks

`#longtabular` uses the same columns, cells, merges, fills, and border controls. It requires a finite `width` and a positive `pageheight`. Every column must be fixed or flex; natural columns are not supported in long tables. The following complete example deliberately forces a break, so it demonstrates both the first and continuation sections.

```
1 #longtabular(width=340pt, pageheight=180pt, grid=all,
2             minheight=24pt, padx=5pt, pady=4pt) {
3   columns {
4     col(width=35pt, align=right);
5     col(width=flex(3));
6     col(width=flex(1), align=right);
7   }
8   firsthead {
9     row { cell(colspan=3, align=center) { Inventory report } }
10    row { cell { ID } cell { Description } cell { Units } }
11  }
12  head {
13    row { cell(colspan=3, align=center) { Inventory continued } }
14    row { cell { ID } cell { Description } cell { Units } }
15  }
16  foot { row { cell(colspan=3, align=right) { Continued... } } }
17  lastfoot { row { cell(colspan=3, align=right) { End of report } } }
18  body {
19    row { cell { 1 } cell { Paper } cell { 20 } }
20    keep {
21      row { cell { 2 } cell { Kit: outer case } cell { 1 } }
22      row { cell { 3 } cell { Kit: inserts } cell { 4 } }
23    }
24    break;
25    row { cell { 4 } cell { New category } cell { 2 } }
26    nobreak;
27    row { cell { 5 } cell { Category subtotal } cell { 2 } }
28    row { cell { 6 } cell { Final item } cell { 1 } }
29  }
30 }
```

Repeated sections. After `columns`, sections may appear once each in any order. `firsthead` is used on the first page and `head` on later pages. `foot` is used on nonfinal pages and `lastfoot` on the final page. Omitted `firsthead` inherits `head`; omitted `lastfoot` inherits `foot`. Omitted `head` and `foot` are empty. An explicit empty section, such as `firsthead {}`, overrides inheritance. Repeated content is boxed once and copied, rather than evaluated on every page.

Keeping rows together. `keep { ... }` groups complete body rows and can contain intervening horizontal rules. Keep blocks cannot nest. Between body rows, `break;` forces a break and `nobreak;` forbids one. Rowspans, keep blocks, and nobreak boundaries form indivisible groups. A forced break inside such a group is an error. A cell paragraph is never split across pages.

The renderer reserves space for the applicable header, footer, and border bands before selecting a nonempty group of rows. If the final footer is taller than the continuation footer, it is included in the final-page fit check. A group that cannot fit is an error; the renderer does not produce an empty footer-only page. Body rules at a page cut appear at both exposed ends. Vertical border patterns restart on each page fragment.

Live output on the next two pages. The following pages are generated by the listing above. The small `pageheight=180pt` is intentional: each fragment occupies a dedicated page, and the remainder of that page is left for the host's page layout. The first footer says "Continued..."; the final footer says "End of report". Documentation resumes on a fresh page after the table.

Inventory report		
ID	Description	Units
1	Paper	20
2	Kit: outer case	1
3	Kit: inserts	4
Continued...		

Inventory continued		
ID	Description	Units
4	New category	2
5	Category subtotal	2
6	Final item	1
End of report		

5 Lua API

Vesti uses Lua in both building script and inline inside of Vesti code. Below are functions which are baked in Vesti compiler.

compile	download	getCurrentDir	getEngineType
getModule	joinpath	mkdir	parse
ping	print	setCurrentDir	unzip
vestiDummyDir			

Table 3: Lua builtin functions in Vesti

6 Source Code of This Document

Below code was generated by inline lua.

```
1 docclass article (10pt)
2 importpkg {
3     geometry (a4paper, margin = 2.2cm),
4     xcolor,
5     tikz,
6     fancyvrb,
7     amsmath,
8 }
9
10 \title{Vesti Transpiler User Manual}
11 \author{Sungbae Jeong}
12 #::
13 local function read_all(path)
14     local f, err = io.open(path, "rb")
15     assert(f, ("cannot open %s: %s"):format(path, err))
16     local data = f:read("*a")
17     f:close()
18     return data
19 end
20 ::#<readAll>
21
22 -- definition of \keyword command
23 defun [!] keyword (m) {{\tt\color{purple}#1}}
24 defun [!] builtin (v) {{\tt\color{blue!65!yellow}\##1}}
25 #::
26 local commands = { "useenv", "defun", "defenv" }
27 for _, command in ipairs(commands) do
28     local s = string.format(
29         "defun %s (s) {{\IfBooleanTF{#1}{\keyword{%%!-%s-%%}}{\keyword{%%!-%s-%%} } }",
30         command, command, command
31     )
32     vesti.print(vesti.parse(s))
33 end
34 ::#
35
36 #::
37 function begenv(add_bang, add_space)
38     local name = "begenv"
39     if add_bang then name = name .. "!" end
40     local begenv = "\\keyword{" .. name .. "}"
41     if add_space then begenv = begenv .. " " end
42     vesti.print(begenv, { nl = 0 })
43 end
44
45 function endenv(add_bang, add_space)
46     local name = "endenv"
47     if add_bang then name = name .. "!" end
48     local endenv = "\\keyword{" .. name .. "}"
49     if add_space then endenv = endenv .. " " end
50     vesti.print(endenv, { nl = 0 })
51 end
52 ::#*
53
54 -- space character used in the textbook.
55 #at_on
56 #chardef 2423 \@b
```

```

57 defun ob {\kern2pt{\tt\@b}\kern2pt}
58 #at_off
59
60 startdoc
61 \maketitle
62 \tableofcontents
63
64 \section{Introduction}
65 I used to create several documents using \LaTeX\ (or plain \TeX\ but \TeX\ is
66 quite cumbersome to write-especially when working with very complex tables or
67 inserting images). Its markdown-like syntax is also not comfortable to use. For
68 example, here is a simple \LaTeX\ document:
69
70 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
71 % coprime is my custom class. See https://github.com/e0328eric/coprime.
72 \documentclass[tikz, geometry]{coprime}
73
74 \settitle{My First Document}{Sungbae Jeong}{ }
75 \setgeometry{a4paper, margin = 2.5cm}
76
77 \begin{document}
78 \section{Foo}
79 Hello, World!
80 \begin{figure}[ht]
81   \centering
82   \begin{tikzpicture}
83     \draw (0,0) -- (1,1);
84   \end{tikzpicture}
85 \end{figure}
86
87 The code above is a figure using TikZ.
88
89 \end{document}
90 -%}
91
92 What annoys me most when using it is the \lq\verb@\begin@\rq\ and
93 \lq\verb@\end@\rq\ blocks. Is there a way to write something much simpler? This
94 question led me to start this project. Currently, the following code is
95 generated into the \LaTeX code above (except comments) using vesti:
96
97 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
98 % coprime is my custom class. See https://github.com/e0328eric/coprime.
99 +keyword|docclass@ coprime (tikz, geometry)
100
101 \settitle{My First Document}{Sungbae Jeong}{ }
102 \setgeometry{a4paper, margin = 2.5cm}
103
104 +keyword|startdoc@
105
106 \section{Foo}
107 Hello, World!
108 +keyword|useenv@ figure [ht] {
109   \centering
110   +keyword|useenv@ tikzpicture {
111     \draw (0,0) -- (1,1);
112   }
113 }
114
115 The code above is a figure using TikZ.
116 -%}

```

```

117
118 Everywhere in this paper, the symbol \ob\ denotes a \lq\lq space\rq\rq\ when
119 one use Vesti.
120
121 \section{Structure of Vesti File}
122 Vesti is similar as \LaTeX. Its structure consists with two parts: {\tt preamble} and
123 {\tt main}. Preamble is the place where \LaTeX\ documentclass, packages, and
124 several settings are located. Main body is where actual documentation is located.
125 Below figure is the simple Vesti documentation.
126
127 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
128 +keyword|docclass@ article (10pt)
129 +keyword|importpkg@ {
130     geometry (a4paper, margin=2.2cm)
131 }
132 +keyword|startdoc@
133 Hello, Vesti!
134 -%}
135
136 We will see later, but the very difference with \LaTeX\ is that Vesti has its
137 own keywords (keywords are colored with purple). It makes the code readable and
138 it is easier and faster to write the document. The keyword startdoc splits
139 the preamble and the main part of the documentation similar with
140
141 -- Don't ask why I chose Q for catcode 0.
142 %#{\tt\catcode`Q=0 Qcatcode`\=12 \beginQ{documentQ}} in \LaTeX.
143 However, Vesti does not have the analogous part of
144 %#{\tt\catcode`Q=0 Qcatcode`\=12 \endQ{documentQ}},
145 because almost every \LaTeX\ document (99.999\% I'm sure) does not have any code
146 below %#{\tt\catcode`Q=0 Qcatcode`\=12 \endQ{documentQ}}.
147 For this reason, Vesti automatically ends document when EOF (End Of File) is
148 found.
149
150 \section{Keywords}
151 Followings are reserved as keywords. In this document, every Vesti keyword has
152 the form like \keyword{this}.
153 useenv table [ht] {
154     \centering
155     #::
156     local content = read_all("../src/lexer/Token.zig")
157
158     -- Lua's built-in patterns don't support lookahead.
159     -- We capture both the keyword and the TokenType, then filter out 'deprecated'.
160     -- Pattern breakdown:
161     --   %.{          => matches ".{"
162     --   %s*"([~"]+)" => a quoted string -> capture 1
163     --   %s*,%s*TokenType%.([%w_]+) => TokenType.<Name> -> capture 2
164     local pat = "%.{%s*"([~"]+)"%s*,%s*TokenType%.([%w_]+)"
165
166     local keywords = {}
167     for name, tok in content:gmatch(pat) do
168         if tok ~= "deprecated" then
169             keywords[#!keywords + 1] = name
170         end
171     end
172
173     table.sort(keywords)
174
175     vesti.print([[begin{tabular}{cccc}]]
176

```

```

177   for i, kw in ipairs(keywords) do
178       local cell = string.format("\\keyword{%s}", kw)
179       if (i % 4) == 0 then
180           vesti.print(cell .. [[\\]])
181       else
182           vesti.print(cell .. "&")
183       end
184   end
185
186   vesti.print([[\\end{tabular}]]))
187   ::#[readAll]
188   \\caption{Keywords in Vesti}
189 }
190
191 \\subsection{\\keyword{docclass} keyword}
192 Keyword \\keyword{docclass} is an analogous of \\verb|\\documentclass| in \\LaTeX.
193 If \\keyword{docclass} is in the main paragraph, it acts just a normal word.
194 In other words, \\keyword{docclass} actives only in the preamble.
195 The syntax of \\keyword{docclass} is following:
196
197 useenv center {
198     \\keyword{docclass}\\ob<class name>\\ob{\\tt{<arguments>}{\\tt}}
199 }
200 Here, arguments are separated by commas and embraced by {\\tt ()}. Here are some
201 examples.
202
203 \\goodbreak
204 useenv itemize {
205     \\item \\keyword{docclass}\\ob{\\tt article}
206     \\item \\keyword{docclass}\\ob{\\tt article\\ob(10pt)}
207     \\item \\keyword{docclass}\\ob{\\tt article\\ob(10pt,\\ob twocols)}
208     \\item \\keyword{docclass}\\ob{\\tt article\\ob(10pt,twocols)}
209 }
210
211 \\subsection{\\keyword{importpkg} keywords}
212 Keyword \\keyword{importpkg} is an analogous of \\verb|\\usepackage| in \\LaTeX.
213 If \\keyword{importpkg} is in the main paragraph, it acts just a normal word.
214 In other words, \\keyword{importpkg} actives only in the preamble.
215
216 importpkg has two different syntax. First one is same as docclass.
217 useenv center {
218     \\keyword{importpkg}\\ob<pkg-name>\\ob{\\tt{arguments}{\\tt}}
219 }
220 Here, arguments are separated by commas and embraced by {\\tt ()}.
221 In the practical case, one should include several packages with options.
222 importpkg also supports such case. We will look at an example instead of
223 giving rigorous grammar.
224 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
225 +keyword|importpkg@ {
226     amsmath, amssymb, amsthm,
227     geometry (a4paper, margin=2.2cm),
228 }
229 -%}
230
231 \\noindent As one can see, inside of \\verb|{}|, several packages can be used
232 together with thier options.
233
234 \\subsection{\\keyword{startdoc} keyword}
235 Keyword \\keyword{startdoc} tells to Vesti that the main document starts. In the
236 main document, you can also write \\keyword{startdoc} in the main document. In

```

```

237 that case, \keyword{startdoc} does nothing.
238
239 \subsection{\useenv keyword}
240 As the name implies, keyword \useenv is an analogous of \verb|\begin{...}| and
241 \verb|\end{...}| pair in \LaTeX.
242 The simplest \useenv is like this.
243
244 useenv figure [ht] {
245     \centering
246     useenv tikzpicture {
247         useenv scope {
248             \path (0,0) node {\vbox{
249                 %#\hbox{\tt\useenv center \{
250                 %#\hbox{\tt\obeyspaces Hello, World!}
251                 %#\hbox{\tt\obeyspaces\}}
252             }};
253         }
254         \path (2.3,0) node {or};
255         useenv scope [shift={(6,0)}]{
256             \path (0,0) node {\tt\useenv center \{ Hello, World! \}};
257         }
258     }
259 }
260
261 As you can see, {\tt\useenv center} is the part of \verb|\begin{center}|, and
262 the single {\tt\}} is the part of \verb|\end{center}|. Since Vesti knows their
263 pair, one can write a code with several environment, and each pair is properly
264 matched. For instance, above example is written in Vesti like follows. Here,
265 \verb|\useenv| just prints \useenv in that style.
266
267 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
268 |+color|purple@useenv@ figure [ht] {
269     \centering
270     |+color|purple@useenv@ tikzpicture {
271         |+color|purple@useenv@ scope {
272             \path (0,0) node {\vbox{
273                 |+color|blue@%#\hbox{\tt\useenv center \{
274                 |+color|blue@%#\hbox{\tt\obeyspaces Hello, World!}
275                 |+color|blue@%#\hbox{\tt\obeyspaces\}}
276             }};
277         }
278         \path (2.3,0) node {or};
279         |+color|purple@useenv@ scope [shift={(6,0)}] {
280             \path (0,0) node {\tt\useenv center \{ Hello, World! \}};
281         }
282     }
283 }
284 -%}
285
286 Full syntax about \useenv is the following.
287 useenv center {
288     \useenv\ob<environment name>\ob<argument>*\ob{\tt\{ <body> {\tt\}}
289 }
290 where `*' means that the number of <argument> is zero or at least one, and
291 $$
292     "<argument>" = useenv cases {
293         "<argument>" & "mandatory arguments" \cr
294         "[<argument>]" & "optional arguments" \cr
295     }
296 $$

```

```

297 For instance, below one is a valid Vesti code (environment {\tt foo} is
298 undefined in general). As one can see, spaces cannot exist in between <argument>s.
299 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
300 |+color|purple@useenv@ foo (asd)(fff)[\ames and \awdsa](askws)[\rrsaa] {
301     foobar
302 }
303 -%}
304
305 #def #begenv {#::begenv(false,true)::#*}
306 #def #begenv_arg {#::begenv(#1,#2)::#*}
307 #def #endenv {#::endenv(false,true)::#*}
308 #def #endenv_arg {#::endenv(#1,#2)::#*}
309
310 \subsection{#begenv and #endenv keywords}
311 As the name implies, both keywords #begenv and #endenv are
312 analogous of \verb|\begin{...}| and \verb|\end{...}| pair in \LaTeX,
313 respectively. Thus below code
314 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
315 |+color|purple@begenv@ center
316     asdsad
317 |+color|purple@endenv@
318 -%}
319 is exactly same as
320 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
321 +keyword|useenv@ center {
322     asdsad
323 }
324 -%}
325 Then why we need #begenv and #endenv if we already have \useenv*?
326 The only reason which both #begenv and #endenv exist is
327 defining new environment. For instance, one defines a new environment like
328 following.
329 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@] {
330 %#+keyword|defenv@ foo { |+color|purple@begenv@ minipage(\textwidth)}{ |+color|purple@endenv@}
331 }
332
333 \subsection{\defun keyword}
334 \defun is the keyword which makes a \LaTeX\ function. Internally, it uses {\tt
335 xparse} package which is automatically included from modern \LaTeX$2\epsilon$
336 kernel. The grammar of \defun is the following.
337
338 begenv center
339     \defun\ob[<attr>]?\ob<command-name>\ob(<param-spec>)?\ob\{<implementation>\}
340 endenv
341 Here, {\tt`?`} means that this is an optional.
342
343 First, let us see some examples for \defun*, and then explain the meaning of
344 each fields.
345
346 #::
347 local examples = {
348     "\defun foo {Hello, World!}"
349 }
350
351 for _, example in ipairs(examples) do
352     vesti.print(vesti.parse(
353         "begenv! Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]"
354     ))
355     vesti.print(example)
356     vesti.print(vesti.parse("endenv! Verbatim"))

```

```

357 end
358 ::#
359
360 \section{Builtins}
361 Vesti also has its own builtin functions, which are prefixed with \#.
362 One might wonder what distinguishes builtins from keywords. In fact, from the
363 compiler's internal perspective, there is no real difference. However, in actual
364 language usage, constantly typing the prefix can be somewhat tedious, especially
365 for functions that are commonly used.
366
367 From the perspective of language design --particularly in Vesti-- it is
368 sometimes desirable to use names that can not be reserved as keywords. For example,
369 Vesti provides a built-in function \builtin|label|, which will be explained
370 later. Since Vesti is a typewriting-oriented language, the word \lq\lq
371 label\rq\rq\ is often used in its ordinary sense rather than in its special
372 semantic meaning within the language.
373
374 Followings are reserved as builtin functions.
375
376 useenv table [ht] {
377     \centering
378     #::
379     local content = read_all("../src/lexer/Token.zig")
380
381     -- match .{ "here" }
382     local pat = "%.{%s*\"([^\"]+)\">%s*%}"
383
384     local builtins = {}
385     for name, tok in content:gmatch(pat) do
386         builtins[#!builtins + 1] = name
387     end
388     table.sort(builtins)
389
390     vesti.print([[ \begin{tabular}{ccccc} ]])
391
392     for i, kw in ipairs(builtins) do
393         local cell = string.format("\\builtin@%s@", kw)
394         if (i % 5) == 0 then
395             vesti.print(cell .. [[ \\ ]])
396         else
397             vesti.print(cell .. "&")
398         end
399     end
400
401     vesti.print([[ \\end{tabular} ]])
402     ::#[readAll]
403     \caption{Builtins in Vesti}
404 }
405 Since {\tt fancyvrb} does not allow verbatim-like commands inside of
406 {\tt Verbatim} environment, builtins are not colored in example codes.
407
408 \subsection{\builtin|label| builtin}
409 For the compiling issue, if one label some environment, one should write like
410 follows using \LaTeX\ function \verb|\label|.
411 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=&|@]{%-
412 &keyword|useenv@ theorem { \label{eq:1}
413     1 + 1 = 2.
414 }
415 -%}
416

```

```

417 However, such code is not natural. Thus \builtin|label| comes into the place.
418 One can label environments using \builtin|label| builtin like this.
419 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=&|@]{%-
420 #label(eq:1) &keyword|useenv@ theorem {
421     1 + 1 = 2.
422 }
423 -%}
424 One can also write like the following, which I personally prefer.
425 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=&|@]{%-
426 #label(eq:1)
427 &keyword|useenv@ theorem {
428     1 + 1 = 2.
429 }
430 -%}
431
432 \subsection{\builtin|eq| builtin}
433 \builtin|eq| is a abbreviation of \lq\keyword{%-useenv-%} equation\rq.
434 Actually, below codes are same.
435
436 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
437 #label(eq:1)
438 +keyword|useenv@ equation {
439     \sum_{n=1}^{\infty} {1}/{n^2} = {\pi^2}/{6}.
440 }
441 -%}
442 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
443 #eq(eq:1) {
444     \sum_{n=1}^{\infty} {1}/{n^2} = {\pi^2}/{6}.
445 }
446 -%}
447 Since Vesti is used in the mathematical documents mainly, small syntactic suger
448 is needed, so Vesti introduce \builtin|eq| builtin.
449
450 \subsection{\builtin|chardef| builtin}
451 \builtin|chardef| defines a text token using the unicode codepoint.
452 Here is the grammar of \builtin|chardef| builtin.
453 useenv center {
454     \builtin|chardef|\ob<unicode-codepoint>\ob<function>
455 }
456 Here, <unicode-codepoint> must be hexadecimal.
457
458 \subsection{\builtin|mathchardef| builtin}
459 \builtin|mathchardef| defines a math token using the unicode codepoint.
460 Here is the grammar of \builtin|mathchardef| builtin.
461 useenv center {
462     \builtin|mathchardef|\ob<kind>\ob<font-num>\ob<unicode-codepoint>\ob<function>
463 }
464 Here, <unicode-codepoint> must be hexadecimal.
465 Before explaining each parameter, introduce some examples.
466 useenv itemize {
467     \item \builtin|mathchardef|\ob.opening\ob0\ob29d8\ob\verb|\lfooo|
468     \item \builtin|mathchardef|\ob.ordinary\ob0\ob2202\ob\verb|\diff|
469 }
470
471 \subsection{\builtin|enum| and \builtin|enum_counter| builtins}
472 \builtin|enum| builtin is just an enumerate environment with minimal support of
473 {\tt enumitem} feature. For example,
474 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
475 #enum {
476     \item aaa

```

```

477     \item bbb
478     \item ccc
479 }
480 -%}
481 prints
482 #enum {
483     \item aaa
484     \item bbb
485     \item ccc
486 }
487 \builtin|enum| also can be nested like follows.
488 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
489 #enum {
490     \item aaa
491     \item #enum {
492         \item bbb
493         \item ccc
494     }
495     \item ddd
496 }
497 -%}
498 It prints
499 #enum {
500     \item aaa
501     \item #enum {
502         \item bbb
503         \item ccc
504     }
505     \item ddd
506 }
507 To customize a format of \verb|\item| like {\tt enumitem}, one can
508 write like the following:
509 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
510 #enum (\Roman*. ) {
511     \item aaa
512     \item #enum ({\roman*}**) {
513         \item bbb
514         \item ccc
515     }
516     \item ddd
517 }
518 -%}
519 #enum (\Roman*. ) {
520     \item aaa
521     \item #enum ({\roman*}**) {
522         \item bbb
523         \item ccc
524     }
525     \item ddd
526 }
527 As one can see, \builtin|enum| changes the format with inside of {\tt()}.
528 In Vesti, single {\tt *} changes into {\tt \{<enum count>\}} (including braces),
529 and consecutive {\tt **} changes into single {\tt *} character.
530 Thus, \verb|\Roman*.| changes into \verb|\Roman{enumi}.| and \verb|{\roman*}**|
531 changes into \verb|{\roman{enumii}}*| (because this part is inside of another
532 \builtin|enum|).
533
534 Beware that by the parting mechanism, \verb|\roman***| changes into
535 \verb|\roman*<enum count>|, and \verb|\roman{*}**| into \verb|\roman{\<enum count>}*|,
536 which are invalid arguments for \verb|\roman| \LaTeX\ internal command.

```

```

537
538 \builtin|enum_counter| builtin is a helper to get the name of the current enum
539 counter. For example, if one wants to start a second enum from 3, one can write
540 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
541 #enum {
542     \item aaa
543     \item #enum {
544         \setcounter{#enum_counter}{2}
545         \item bbb
546         \item ccc
547     }
548     \item ddd
549 }
550 -%}
551 and it prints
552 #enum {
553     \item aaa
554     \item #enum {
555         \setcounter{#enum_counter}{2}
556         \item bbb
557         \item ccc
558     }
559     \item ddd
560 }
561
562 \subsection{\builtin|get_filepath| builtin}
563 Because of the Vesti internal, if one includes picture by using
564 \verb|\includegraphics| for instance, providing the raw path cause an error from
565 \LaTeX\ with \lq\lq file not found\rq\rq. \builtin|get_filepath| builtin adjust
566 raw file path into the new relative one which \LaTeX\ knows. Here is the example
567 code.
568 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@] {
569 %#\includegraphics{#get_filepath(./foo.png)}
570 }
571 Internally, \builtin|get_filepath| changes file path into the relative one with
572 respect to {\tt.vesti-dummy}. For example, if {\tt foo.png}, {\tt foo.ves}
573 and {\tt.vesti-dummy} are located in the root directory, then inside of {\tt
574 foo.ves}, \builtin|get_filepath|{\tt(./foo.png)} will changed into
575 {\tt../foo.png} because for {\tt.vesti-dummy}, {\tt foo.png} is located outside
576 of it.
577
578 \subsection{\builtin|at_on| and \builtin|at_off| builtins}
579 Both builtins are same as \LaTeX\ functions \verb|\makeatletter| and
580 \verb|\makeatother| plus changes Vesti lexer such that \verb|@| character is
581 also can be a \LaTeX\ function name. For instance, without using \builtin|at_on|
582 command, below code does not compiles.
583 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|&] {
584 %#+keyword|defun& @foo (#1) {Hello, #1!}
585 }
586 This is because \keyword{% -defun-%} expects a valid \LaTeX\ function name but
587 the character \verb|@| is not a valid one except after the \verb|\makeatletter|
588 function. Since \keyword{% -defun-%} is a Vesti grammar, there is no way to tell
589 Vesti that \verb|@| is a right one although one uses \verb|\makeatletter|.
590 By using \builtin|at_on|, below code then compiles.
591 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|&]{%-
592 #at_on
593 +keyword|defun& @foo (#1) {Hello, #1!}
594 #at_off
595 -%}
596 One can not using \builtin|at_off|, but I strongly recommend to match

```

```

597 pairs.
598
599 \subsection{\buitin|ltx3_on| and \buitin|ltx3_off| builtins}
600 Those builtins are similar with \buitin|at_on| and \buitin|at_off|
601 pairs but for \LaTeX3.
602
603 \subsection{\buitin|textmode| and \buitin|mathmode| builtins}
604 Internally, vesti tracks so called {\tt text mode} and {\tt math mode}.
605 In the math mode, for instance, the token \verb|>| is changed into $->$ inside
606 of the math mode. The idea is that Vesti changes \verb|>| into \verb|\to|, and
607 so on. But when defining some function using \defun*, for example, one might
608 want to define
609 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
610 +keyword|useenv@ foo {-><-}
611 -%}
612 so that \verb|$foo$| makes $-><-$, one should change the mode. Here,
613 \buitin|mathmode| comes into the place. One should write instead that
614 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
615 +keyword|useenv@ foo {#mathmode{-><-}}
616 -%}
617
618 \clearpage
619 \subsection{\buitin|tabular| and \buitin|longtabular| builtins}
620 \label{sec:native-tables}
621 These builtins make tables directly from \TeX\ boxes, dimensions, glue, and
622 rules. They do not import any \LaTeX\ table, color, or graphics package.
623 \buitin|tabular| produces one indivisible table; \buitin|longtabular|
624 continues a table across dedicated pages with repeated headers and footers.
625 Both support paragraph cells, rectangular merges, colored backgrounds, and
626 six border patterns. Existing \keyword{%-useenv-%} tabular code keeps its
627 usual \LaTeX\ meaning.
628
629 \subsubsection{A complete short table}
630 The table body contains explicit rows and cells. This example includes a
631 spanning title and two colored cells. Its output appears below the listing.
632
633 useenv Verbatim [numbers=left, numbersep=5pt, frame=single]{%-
634 #tabular(width=300pt, grid=all, padx=5pt,
635         pady=4pt, minheight=24pt) {
636     columns {
637         col(width=flex(1), align=center);
638         col(width=flex(1), align=center);
639         col(width=flex(1), align=center);
640     }
641     body {
642         row { cell(colspan=3) { AT\&T Common Stock } }
643         row {
644             cell { Year }
645             cell(background=cmyk(0,.6,.1,0)) { Price }
646             cell { Dividend }
647         }
648         row {
649             cell(background=cmyk(.6,.1,0,0)) { 1971 }
650             cell { 41--!54 } cell { \$2.60 }
651         }
652         row { cell { 1971 } cell { 41--!54 } cell { \$2.60 } }
653     }
654 }
655 -%}
656

```

```

657 #tabular(width=300pt, grid=all, padx=5pt,
658         pady=4pt, minheight=24pt) {
659     columns {
660         col(width=flex(1), align=center);
661         col(width=flex(1), align=center);
662         col(width=flex(1), align=center);
663     }
664     body {
665         row { cell(colspan=3) { AT\&T Common Stock } }
666         row {
667             cell { Year }
668             cell(background=cmyk(0,.6,.1,0)) { Price }
669             cell { Dividend }
670         }
671         row {
672             cell(background=cmyk(.6,.1,0,0)) { 1971 }
673             cell { 41--!54 } cell { \$2.60 }
674         }
675         row { cell { 1971 } cell { 41--!54 } cell { \$2.60 } }
676     }
677 }
678
679 Write \texttt{columns} first and specify \texttt{width} on every \texttt{col}.
680 There must be exactly one nonempty \texttt{body} section. A short table accepts
681 only that section. Write \verb|cell {}| for an intentional empty cell.
682 Newlines and indentation outside cell content have no structural meaning.
683 An ampersand does not separate cells, and a double backslash does not end a row.
684 Use ordinary escaped text such as \verb|\&| and \verb|\$| inside a cell.
685
686 %-Vesti's comment syntax still applies: \verb|--| starts a comment. For a numeric
687 range, write \verb|41--!54| or use a literal en dash. The \verb|--!| token
688 produces the en dash shown in the example.-%
689
690 \clearpage
691 \subsubsection{Options, widths, and alignment}
692 Options have the form \texttt{(key=value, key=value)}; a trailing comma is
693 allowed. Unknown or duplicate keys, an empty option list, and values of the
694 wrong type are errors. The following reference table is itself a native table.
695
696 #tabular(width=\hsize, grid=rows, padx=5pt, pady=4pt, valign=top) {
697     columns { col(width=95pt); col(width=flex(1)); }
698     body {
699         row(before={\bf}) { cell { Scope } cell { Options and defaults } }
700         row {
701             cell { Both builtins }
702             cell {
703                 \texttt{width=natural}, \texttt{align=center},
704                 \texttt{grid=none}, \texttt{padx=4pt}, \texttt{pady=3pt},
705                 \texttt{minheight=0pt}, \texttt{valign=middle}, plus border defaults.
706             }
707         }
708         row {
709             cell { Long table }
710             cell { Requires finite \texttt{width} and \texttt{pageheight};
711                 every column must have a fixed or flex width. }
712         }
713         row {
714             cell { Column }
715             cell { Required \texttt{width}; \texttt{align=left}; optional
716                 \texttt{valign}, \texttt{wrap}, and \texttt{before}. }

```

```

717     }
718     row {
719         cell { Row }
720         cell { Optional \texttt{minheight}, \texttt{background}, and
721             \texttt{before}. Minimum height inherits from the table. }
722     }
723     row {
724         cell { Cell }
725         cell { \texttt{colspan=1}, \texttt{rowspan=1}; optional
726             \texttt{align}, \texttt{valign}, \texttt{wrap},
727             \texttt{background}, and \texttt{before}. }
728     }
729 }
730 }
731
732 \paragraph{Choosing widths.}
733 A column width is \texttt{natural}, a positive dimension such as
734 \texttt{35pt}, or \texttt{flex(positive weight)}. Natural columns measure
735 unwrapped content; fixed and flex columns wrap by default. Set
736 \texttt{wrap=yes} or \texttt{wrap=no} to override this behavior.
737 A wrapped span must include at least one fixed or flex column so its width
738 can be resolved before paragraph typesetting.
739
740 The table's width is \texttt{natural}, a positive dimension, or the register
741 \verb|\hsize| or \verb|\vsize|. Flex columns require a finite table width and
742 divide the remaining space in proportion to their weights. For example, weights
743 1 and 2 share the available space in a ratio of one to two. With no flex columns,
744 an explicit table width must equal the total resolved width within one scaled
745 point; it does not add extra space around fixed columns.
746
747 The \texttt{pageheight} option is available only on \builtin|longtabular|.
748 It accepts a positive dimension or \verb|\hsize| or \verb|\vsize|;
749 \texttt{natural} and \texttt{flex} are not valid page heights.
750
751 A column track includes left and right padding. Vertical borders occupy
752 additional lanes. A colspan includes the internal lanes it replaces.
753 Double borders reserve space for both strokes and their gap. If the target
754 width leaves no positive content width, or an unwrapped cell is too wide,
755 the renderer reports an error instead of enlarging the table silently.
756
757 \paragraph{Alignment and local styling.}
758 Horizontal alignment is \texttt{left}, \texttt{center}, or \texttt{right}.
759 The table's \texttt{align} positions the entire table. Cells inherit alignment
760 and wrapping from their starting column. Vertical alignment is \texttt{top},
761 \texttt{middle}, or \texttt{bottom}. A cell uses its explicit value, then its
762 starting column's value, then the table default.
763 Minimum height is a lower bound and never clips content.
764
765 Use \verb|before={\bf}| for a local font change. Hooks run in this order:
766 starting column, row, cell, then cell content. A cell uses its own background,
767 then its originating row's background; if neither supplies one, it is transparent.
768 A rowspan retains that background across its complete rectangle.
769
770 \paragraph{Dimensions and colors.}
771 Dimensions are unsigned decimals immediately followed by a unit:
772 \texttt{pt}, \texttt{pc}, \texttt{in}, \texttt{bp}, \texttt{cm},
773 \texttt{mm}, \texttt{dd}, \texttt{cc}, \texttt{sp}, \texttt{em}, or
774 \texttt{ex}. For example, \texttt{.6pt} and \texttt{3em} are valid;
775 signs, exponent notation, expressions, and arbitrary macros are not.
776 Registers and font-relative units are resolved at table entry, before cell

```

```

777 font hooks. Widths, page height, stroke thickness, explicit gaps, and dash
778 lengths must be positive. Padding, minimum heights, and pattern phase may be zero.
779
780 Colors are \texttt{gray(g)}, \texttt{rgb(r,g,b)}, or
781 \texttt{cmyk(c,m,y,k)}, with every component between zero and one.
782 There are no named colors or transparency values in the table grammar.
783
784 \clearpage
785 \subsubsection{Merges and styled borders}
786 Use \texttt{colspan} and \texttt{rowspan} with positive integer values to
787 merge a rectangle. Cells are placed in the next free column from left to right.
788 Later rows omit positions already occupied by a rowspan. Spans cannot intersect
789 or cross a section boundary, and all remaining positions must be filled.
790
791 The \texttt{grid} option accepts \texttt{none}, \texttt{frame},
792 \texttt{rows}, \texttt{columns}, or \texttt{all}. The \texttt{rows} and
793 \texttt{columns} values draw only interior boundaries; \texttt{frame} draws
794 the perimeter, and \texttt{all} draws both. Automatic borders inside merged
795 cells are suppressed. Explicit rules must also avoid the interior of a merge.
796
797 This example uses all six border patterns. The partial dotted rule covers only
798 columns two and three, so it leaves the rowspan intact.
799
800 useenv Verbatim [numbers=left, numbersep=5pt, frame=single]{%-
801 #tabular(width=340pt, grid=all, pady=4pt, minheight=24pt) {
802   columns {
803     col(width=flex(1), align=center);
804     col(width=flex(1), align=center);
805     col(width=flex(2));
806   }
807   body {
808     hline(style=solid, thickness=1pt, color=rgb(.1,.2,.4));
809     row { cell(colspan=3) { Merges and border patterns } }
810     hline(style=double, thickness=.7pt, gap=1pt,
811           color=rgb(.1,.3,.6));
812     row {
813       cell(rowspan=2, background=rgb(.9,.95,1)) { Group A }
814       cell { Dotted }
815       cell { A partial rule below this row. }
816     }
817     hline(from=2, to=3, style=dotted, thickness=.7pt,
818           gap=1.2pt, color=rgb(.65,.15,.2));
819     row { cell { Dashed } cell { The merged cell continues. } }
820     hline(style=dashed, dashlength=4pt, gap=2pt,
821           color=rgb(.1,.45,.3));
822     row { cell { B } cell { Dash-dot } cell { Separate marks. } }
823     hline(style=dashdot, dashlength=4pt, gap=1.5pt,
824           phase=.5pt, color=rgb(.35,.2,.6));
825     row { cell { C } cell { Dash-dot-dot } cell { Square dots. } }
826     hline(style=dashdotdot, dashlength=4pt, gap=1.5pt,
827           color=rgb(.6,.3,.1));
828     vline(after=0, style=double, thickness=.7pt, gap=1pt);
829     vline(after=3, style=solid, thickness=1pt);
830   }
831 }
832 -%}
833
834 #tabular(width=340pt, grid=all, pady=4pt, minheight=24pt) {
835   columns {
836     col(width=flex(1), align=center);

```

```

837     col(width=flex(1), align=center);
838     col(width=flex(2));
839 }
840 body {
841     hline(style=solid, thickness=1pt, color=rgb(.1,.2,.4));
842     row { cell(colspan=3) { Merges and border patterns } }
843     hline(style=double, thickness=.7pt, gap=1pt,
844           color=rgb(.1,.3,.6));
845     row {
846         cell(rowspan=2, background=rgb(.9,.95,1)) { Group A }
847         cell { Dotted }
848         cell { A partial rule below this row. }
849     }
850     hline(from=2, to=3, style=dotted, thickness=.7pt,
851           gap=1.2pt, color=rgb(.65,.15,.2));
852     row { cell { Dashed } cell { The merged cell continues. } }
853     hline(style=dashed, dashlength=4pt, gap=2pt,
854           color=rgb(.1,.45,.3));
855     row { cell { B } cell { Dash-dot } cell { Separate marks. } }
856     hline(style=dashdot, dashlength=4pt, gap=1.5pt,
857           phase=.5pt, color=rgb(.35,.2,.6));
858     row { cell { C } cell { Dash-dot-dot } cell { Square dots. } }
859     hline(style=dashdotdot, dashlength=4pt, gap=1.5pt,
860           color=rgb(.6,.3,.1));
861     vline(after=0, style=double, thickness=.7pt, gap=1pt);
862     vline(after=3, style=solid, thickness=1pt);
863 }
864 }
865
866 \clearpage
867 \subsubsection{Border controls and ranges}
868 Table-level defaults apply to automatic grid edges and are inherited by explicit
869 rules. A rule can override each component independently.
870
871 #tabular(width=\hsize, grid=rows, padx=5pt, pady=4pt, valign=top) {
872     columns { col(width=137pt); col(width=90pt); col(width=flex(1)); }
873     body {
874         row(before={\bf}) {
875             cell { Table default } cell { Rule option } cell { Meaning }
876         }
877         row { cell { \texttt{rulestyle=solid} } cell { \texttt{style} }
878              cell { Pattern: solid, double, dotted, dashed, dashdot, dashdotdot. } }
879         row { cell { \texttt{rulewidth=.4pt} } cell { \texttt{thickness} }
880              cell { Width of one stroke. } }
881         row { cell { \texttt{rulecolor=gray(0)} } cell { \texttt{color} }
882              cell { Gray, RGB, or CMYK color. } }
883         row { cell { \texttt{rulegap=auto} } cell { \texttt{gap} }
884              cell { Clear space between marks or double strokes. } }
885         row { cell { \texttt{ruledashlength=auto} }
886              cell { \texttt{dashlength} } cell { Length of each dash. } }
887         row { cell { \texttt{rulephase=0pt} } cell { \texttt{phase} }
888              cell { Distance into the repeating pattern at a run's start. } }
889     }
890 }
891
892 \paragraph{Horizontal and vertical rules.}
893 \texttt{hline;} draws a rule at its position between rows, or before the first
894 or after the last row. Optional \texttt{from} and \texttt{to} select a
895 one-based, inclusive column range; omission means the full width.
896 \texttt{vline(after=0);} selects the left edge. The required \texttt{after}

```

897 numbers column boundaries from zero through the number of columns. Its optional
 898 `\texttt{from}` and `\texttt{to}` select a one-based, inclusive row range in the
 899 current section. A vertical rule's source position does not select its rows.
 900 Omitting both range endpoints selects every row in the current section.
 901
 902 Explicit rules replace automatic grid edges. Identical explicit overlaps merge;
 903 incompatible overlaps are errors, including at header/body/footer joins.
 904 Do not draw a full-width horizontal rule through a `rowspan`: select only its
 905 exposed columns as in the dotted example above.
 906
 907 `\paragraph{Pattern dimensions.}`
 908 For final stroke thickness $\$t\$$, `\texttt{gap=auto}` means $\$t\$$ for double lines
 909 and $\$2t\$$ for repeating patterns. `\texttt{dashlength=auto}` means $\$4t\$$.
 910 A double border reserves $\$2t+g\$$, where $\$g\$$ is its clear gap; all other patterns
 911 reserve $\$t\$$. Changing thickness or a double gap can change layout and page fits.
 912 Changing color, dash spacing, or phase alone does not change the reserved width.
 913 An explicit local `\texttt{auto}` resets an inherited numeric value.
 914
 915 On an individual rule, `\texttt{gap}` applies to nonsolid styles;
 916 `\texttt{dashlength}` applies only to dashed, `dashdot`, and `dashdotdot`;
 917 `\texttt{phase}` applies only to repeating patterns. Inapplicable explicit
 918 controls are errors. Table defaults may remain unused until a matching style
 919 uses them.
 920
 921 Dots are square rules, not font characters. Dash-dot repeats dash, gap, dot,
 922 gap; dash-dot-dot adds one more dot and gap. Patterns run left to right or top
 923 to bottom, normalize phase modulo the full period, and clip at their endpoints.
 924 Equal adjacent borders form one continuous run across ordinary row/column
 925 junctions. A merge interruption, style change, or page cut starts a new run.
 926 Clear gaps are transparent; horizontal ink paints over vertical ink at crossings.
 927
 928 `\clearpage`
 929 `\subsubsection{Long tables and page breaks}`
 930 `\builtin|longtabular|` uses the same columns, cells, merges, fills, and border
 931 controls. It requires a finite `\texttt{width}` and a positive
 932 `\texttt{pageheight}`. Every column must be fixed or flex; natural columns are
 933 not supported in long tables. The following complete example deliberately
 934 forces a break, so it demonstrates both the first and continuation sections.
 935
 936 `useenv Verbatim [numbers=left, numbersep=5pt, frame=single]{%-`
 937 `#longtabular(width=340pt, pageheight=180pt, grid=all,`
 938 `minheight=24pt, padx=5pt, pady=4pt) {`
 939 `columns {`
 940 `col(width=35pt, align=right);`
 941 `col(width=flex(3));`
 942 `col(width=flex(1), align=right);`
 943 `}`
 944 `firsthead {`
 945 `row { cell(colspan=3, align=center) { Inventory report } }`
 946 `row { cell { ID } cell { Description } cell { Units } }`
 947 `}`
 948 `head {`
 949 `row { cell(colspan=3, align=center) { Inventory continued } }`
 950 `row { cell { ID } cell { Description } cell { Units } }`
 951 `}`
 952 `foot { row { cell(colspan=3, align=right) { Continued... } } }`
 953 `lastfoot { row { cell(colspan=3, align=right) { End of report } } }`
 954 `body {`
 955 `row { cell { 1 } cell { Paper } cell { 20 } }`
 956 `keep {`

```

957     row { cell { 2 } cell { Kit: outer case } cell { 1 } }
958     row { cell { 3 } cell { Kit: inserts } cell { 4 } }
959 }
960 break;
961 row { cell { 4 } cell { New category } cell { 2 } }
962 nobreak;
963 row { cell { 5 } cell { Category subtotal } cell { 2 } }
964 row { cell { 6 } cell { Final item } cell { 1 } }
965 }
966 }
967 -%}
968
969 \paragraph{Repeated sections.}
970 After \texttt{columns}, sections may appear once each in any order.
971 \texttt{firsthead} is used on the first page and \texttt{head} on later pages.
972 \texttt{foot} is used on nonfinal pages and \texttt{lastfoot} on the final page.
973 Omitted \texttt{firsthead} inherits \texttt{head}; omitted \texttt{lastfoot}
974 inherits \texttt{foot}. Omitted \texttt{head} and \texttt{foot} are empty.
975 An explicit empty section, such as \verb|firsthead {}|, overrides inheritance.
976 Repeated content is boxed once and copied, rather than evaluated on every page.
977
978 \paragraph{Keeping rows together.}
979 \verb|keep { ... }| groups complete body rows and can contain intervening
980 horizontal rules. Keep blocks cannot nest. Between body rows, \texttt{break;}
981 forces a break and \texttt{nobreak;} forbids one. Rowspans, keep blocks, and
982 nobreak boundaries form indivisible groups. A forced break inside such a group
983 is an error. A cell paragraph is never split across pages.
984
985 The renderer reserves space for the applicable header, footer, and border bands
986 before selecting a nonempty group of rows. If the final footer is taller than
987 the continuation footer, it is included in the final-page fit check. A group
988 that cannot fit is an error; the renderer does not produce an empty footer-only
989 page. Body rules at a page cut appear at both exposed ends. Vertical border
990 patterns restart on each page fragment.
991
992 \paragraph{Live output on the next two pages.}
993 The following pages are generated by the listing above. The small
994 \texttt{pageheight=180pt} is intentional: each fragment occupies a dedicated
995 page, and the remainder of that page is left for the host's page layout.
996 The first footer says ``Continued...''; the final footer says ``End of report''.
997 Documentation resumes on a fresh page after the table.
998
999 \clearpage
1000 #longtabular(width=340pt, pageheight=180pt, grid=all,
1001             minheight=24pt, padx=5pt, pady=4pt) {
1002   columns {
1003     col(width=35pt, align=right);
1004     col(width=flex(3));
1005     col(width=flex(1), align=right);
1006   }
1007   firsthead {
1008     row { cell(colspan=3, align=center) { Inventory report } }
1009     row { cell { ID } cell { Description } cell { Units } }
1010   }
1011   head {
1012     row { cell(colspan=3, align=center) { Inventory continued } }
1013     row { cell { ID } cell { Description } cell { Units } }
1014   }
1015   foot { row { cell(colspan=3, align=right) { Continued... } } }
1016   lastfoot { row { cell(colspan=3, align=right) { End of report } } }

```

```

1017 body {
1018   row { cell { 1 } cell { Paper } cell { 20 } }
1019   keep {
1020     row { cell { 2 } cell { Kit: outer case } cell { 1 } }
1021     row { cell { 3 } cell { Kit: inserts } cell { 4 } }
1022   }
1023   break;
1024   row { cell { 4 } cell { New category } cell { 2 } }
1025   nobreak;
1026   row { cell { 5 } cell { Category subtotal } cell { 2 } }
1027   row { cell { 6 } cell { Final item } cell { 1 } }
1028 }
1029 }
1030
1031 \section{Lua API}
1032 Vesti uses Lua in both building script and inline inside of Vesti code.
1033 Below are functions which are baked in Vesti compiler.
1034 useenv table [ht] {
1035   \centering
1036   #::
1037   local content = read_all("../src/Lua.zig")
1038
1039   local pat = '%.name%s*=%s*"([~"]+)"'
1040
1041   local lua = {}
1042   for name, tok in content:gmatch(pat) do
1043     lua[#lua + 1] = name
1044   end
1045   table.sort(lua)
1046
1047   vesti.print([[ \begin{tabular}{ccccc} ]])
1048
1049   for i, kw in ipairs(lua) do
1050     local cell = string.format("\\tt{%s}", kw)
1051     if (i % 4) == 0 then
1052       vesti.print(cell .. [[ \\ ]])
1053     else
1054       vesti.print(cell .. "&")
1055     end
1056   end
1057
1058   vesti.print([[ \\end{tabular} ]])
1059   ::#[readAll]
1060   \caption{Lua builtin functions in Vesti}
1061 }
1062
1063
1064 \newpage
1065 \section{Source Code of This Document}
1066 Below code was generated by inline lua.
1067 useenv Verbatim [numbers=left, numbersep=5pt, frame=single] {
1068   #::
1069   local content = read_all("vesti_man.ves")
1070   for line in content:gmatch("(^[~r~n]*)\\r?\\n?") do
1071     vesti.print(line)
1072   end
1073   ::#[readAll]
1074 }
1075

```